Type-checking format args

Document #:	P2757R1
Date:	2023-03-13
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2757R0], reverted `basic_format_parse_context` constructor and removed `check_dynamic_spec_arithmetic` - "arithmetic" types technically include `bool` and `char` per the language wording, but those are very unlikely to be actually desired in the context where you're asking for something that could also be an `int` or a `double`. Can always be added back in some form if dynamic floating point argument use-cases surface.

§ 2 Introduction

`std::format` supports compile-time checking of format strings [P2216R3], which is a fantastic feature. A compile-time error is always better than a runtime error, and we can see that happen in a lot of cases:

expression	result
`format("{:d}", "I am not a number")`	compile error (invalid specifier for strings)
`format("{:7^*}", "hello")`	compile error (should be `*^7`)
`format("{:>10}", "hello")`	ok
`format("{0:>{1}}", "hello", 10)`	ok
`format("{0:>{2}}", "hello", 10)`	compile error (argument `2` is out of bounds)

expression	result
`format("{:>{}}", "hello")`	compile error (missing an argument for dynamic width)
`format("{:>{}}", "hello", "10")`	runtime error

Wait, why is the last one a runtime error instead of compile-time error?

§ 2.1 The parse context

`formatter<T>::parse` gets an instance of `basic_format_parse_context`, which looks like this (22.14.6.5 [format.parse.ctx]):

```
namespace std {
  template<class charT>
  class basic_format_parse_context {
  public:
    using char_type = charT;
    using const_iterator = typename
        basic_string_view<charT>::const_iterator;
    using iterator = const_iterator;

  private:
    iterator begin_;                                    // exposition only
    iterator end_;                                      // exposition only
    enum indexing { unknown, manual, automatic };       // exposition only
    indexing indexing_;                                 // exposition only
    size_t next_arg_id_;                                // exposition only
    size_t num_args_;                                   // exposition only

  public:
    constexpr explicit basic_format_parse_context(basic_string_view<charT>
        fmt,
                                                  size_t num_args = 0)
        noexcept;
    basic_format_parse_context(const basic_format_parse_context&) = delete;
    basic_format_parse_context& operator=(const basic_format_parse_context&)
        = delete;

    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr void advance_to(const_iterator it);

    constexpr size_t next_arg_id();
    constexpr void check_arg_id(size_t id);
  };
}
```

The user facing logic here is divided into two parts:

— access to the format string itself (`begin()`, `end()`, and `advance_to(it)`)

— access to the arguments (`next_arg_id()` and `check_arg_id(id)`)

Note that the parse context here doesn't get access to the arguments themselves, it only knows how many arguments there are and, if doing automatic indexing, what the current argument index is. This portion of the API can be used to validate that dynamic arguments *exist* (ensuring that two of the rows above fail) and, for automatic indexing, storing the argument index for future access in `formatter<T>::format`.

The parse context doesn't get access to the arguments largely for code size reasons, and also because now that `parse()` is invoked during constant evaluation time, it's unlikely or simply impossible to provide the arguments at that time anyway.

But this API has the limitation that it cannot currently allow diagnosing that last line:

```
format("{:>{}}", "hello", "10")
```

Here, the issue is that we have a dynamic width (the `{}` part), which refers to the next argument, which is `"10"`. But for `char const*`, the width needs to be integral, which `"10"` is not. Now, we don't need to know the *value* of the argument in order to reject this case - we only need to know the type. Which we definitely have. So maybe we can do better?

§ 2.2 Implementation in `{fmt}`

The `{fmt}` library actually *does* reject this example at compile time. It does so by constructing a different kind of parse context that is only used at compile time: the appropriately-named `compile_parse_context`. This is a `basic_format_parse_context` that additionally stores information about what *types* the arguments are, except type-erased to the set that of types that is correctly stored in the `variant` in `basic_format_context`.

The relevant API of `compile_parse_context` looks like this (in `{fmt}`, `basic_format_parse_context` has a second template parameter that is the error handler. It's not relevant for this example. The rest of the code is slightly altered for paper-ness):

```
enum class type {
    none_type,
    // Integer types should go first,
    int_type,
    uint_type,
    long_long_type,
    ulong_long_type,
    int128_type,
    uint128_type,
    bool_type,
    char_type,
    last_integer_type = char_type,
    // followed by floating-point types.
    float_type,
    double_type,
    long_double_type,
    last_numeric_type = long_double_type,
    cstring_type,
    string_type,
    pointer_type,
    custom_type
```

```
};

constexpr auto is_integral_type(type t) -> bool {
    return t > type::none_type && t <= type::last_integer_type;
}

template <typename Char, typename ErrorHandler>
class basic_format_parse_context : private ErrorHandler {
public:
    // these are the same as in std
    constexpr auto next_arg_id() -> int;
    constexpr auto check_arg_id(int arg_id) -> void;

    // but this one is new
    constexpr auto check_dynamic_spec(int arg_id) -> void;
};

template <typename Char, typename ErrorHandler>
class compile_parse_context : basic_format_parse_context<Char, ErrorHandler>
        {
    std::span<type const> types_;

public:
    constexpr auto arg_type(int id) const -> type { return types_[id]; }

    constexpr auto check_dynamic_spec(int arg_id) -> void {
        if (arg_id < types_.size() and not is_integral_type(types_[arg_id]))
        {
            // this ensures that the call is not a constant expression
            this->on_error("width/precision is not an integer");
        }
    }
};

template <typename Char, typename ErrorHandler>
constexpr auto basic_format_parse_context<Char,
        ErrorHandler>::check_dynamic_spec(int arg_id) -> void {
    if consteval {
        using compile_context = compile_parse_context<Char, ErrorHandler>;
        static_cast<compile_context*>(this)->check_dynamic_spec(arg_id);
    }
}
```

There are several important things to note here.

First, the implementation is the only one constructing the parse context, so it's free to do something like - construct a `compile_parse_context` if during constant evaluation time so that this downcast is safe.

Second, the type check *only* happens during constant evaluation time. This is important. In typical uses, `parse` will be called twice: once during compile time (for initial type checking) and then once later during runtime. If we already did the type check during compile time, we don't have to do it *again* during runtime. The conditional checking during `if consteval` is the right way to go.

Third, `{fmt}` uses an enum type that maps all user-defined types to `custom_type`. This is exposed to the user via `check_dynamic_spec` (which checks that the argument type is integral) and `arg_type` (which simply returns the enum). There is no user-provided code being run here - which is important because that lets us basically hide this check behind compile time and not have to worry about whether some arbitrary user-defined predicate is being run or not. It also means that users don't have to worry about the potential overhead of these checks, since they can just choose to call `check_dynamic_spec` and know that this has no runtime overhead – rather than them having to write `if consteval` (and probably forget to).

Finally, because `compile_parse_context` inherits from `basic_format_parse_context`, implementations of `formatter<T>::parse` can still happily take a `basic_format_parse_context<char>&` and continue to work. It's just that now, during compile time, the dynamic type of that context will be different. This means we can add this functionality without breaking user code or requiring the user to make any other changes.

Note that even here, `compile_parse_context` doesn't have the *actual* format arguments - just their types.

§ 2.3 The constructor for `basic_format_parse_context`

Currently, we specify a constructor for `basic_format_parse_context`, though we don't do the same for `basic_format_context`. Only the implementation should be constructing a `basic_format_parse_context` anyway - the constructor we do specify doesn't let us propagate the state properly, and the thing isn't copyable or movable. The constructor is a bit problematic in that its presence would seem to require specifying just how all this type information from the arguments is encoded.

However, actually using this constructor in a way that requires reading arguments is inherently problematic - as the user has no way of providing those arguments in the future. Using this constructor *just* to parse a format string is at least potentially usable:

Parse during `format`	Parse during `parse`

`template <>`
`struct std::formatter<PointHex>`
`  {`
`  constexpr auto parse(auto&`
`  ctx) {`
`  return ctx.begin();`
`  }`
 	
`  auto format(PointHex p, auto&`
`  ctx) const {`
`  return std::format_to(`
`  ctx.out(),`
`  "(x={:x}, y={:x})",`
`  p.x,`
`  p.y`
`  );`
`  }`
`};`
	`template <>`
	`struct std::formatter<PointHex> {`
	`  std::formatter<int> f;`
	
	`  constexpr auto parse(auto& ctx) {`
	`  std::format_parse_context c("x");`
	`  if (f.parse(c) != c.end()) {`
	`  throw std::format_error("wat");`
	`  }`
	
	`  return ctx.begin();`
	`  }`
	
	`  auto format(PointHex p, auto& ctx) const {`
	`  ctx.advance_to(std::format_to(ctx.out(), "`
	`  (x="));`
	`  ctx.advance_to(f.format(p.x, ctx));`
	`  ctx.advance_to(std::format_to(ctx.out(), ",`
	`  y="));`
	`  ctx.advance_to(f.format(p.y, ctx));`

Parse during `format`	Parse during `parse`
	ctx.advance_to(std::format_to(ctx.out(), ")")); return ctx.out(); } };

The latter implementation is significantly more tedious, but only requires parsing the format string for the `int` once. This is something that somebody might actually write, so it needs to stay supported. But this is really only useful in the case where the “fake” parse context has no arguments - which is happily the case where we also don’t have to worry about how to propagate type information for those arguments, since there aren’t any.

3 Proposal

In `{fmt}`, we have `check_dynamic_spec(int)`. This is sufficient for all the standard types - for whom a dynamic spec is integral, and that’s the only thing you’d want to check. But user-defined types might have arbitrary other dynamic parameters, which need not be integral themselves. So the user will need to specify what the allowed types are somehow - in a way that doesn’t require an arbitrary predicate (since we want to avoid the question of dealing with side effects).

There’s basically two ways of doing this:

1. Expose an `enum`, similar to `fmt::detail::type`, and add a function like:

```
constexpr auto check_dynamic_spec(int, std::initializer_list<format_type>) -
        > void;
```

2. Don’t expose an `enum`, instead make this a function template (the implementation would then convert those types into the corresponding enum anyway):

```
template <typename... Ts>
constexpr auto check_dynamic_spec(int) -> void;
```

In both cases, this function only has effects during constant evaluation time - and the only effect is to force a compile error. Either way, we can then also, for convenience, provide a few helpers for all the common cases:

```
// for int, unsigned int, long long int, unsigned long long int
constexpr auto check_dynamic_spec_integral(int) -> void;
// for const char_type* and basic_string_view<char_type>
constexpr auto check_dynamic_spec_string(int) -> void;
```

These both have clear use-cases: dynamic width or precision for the former, dynamic delimiter for the latter.

The enum approach requires specifying an enum. The template approach, if users make their `formatter<T>::parse` a function template (which is going to be the common case, especially since you can just write `auto&`), requires writing `.template` (which is… still shorter, but also awful):

```
ctx.check_dynamic_spec(id, {std::format_type::char_type});

ctx.template check_dynamic_spec<char>(id);
```

This paper proposes the template approach.

3.1 Wording

Add to 22.14.6.5 [format.parse.ctx]:

```
namespace std {
  template<class charT>
  class basic_format_parse_context {
  public:
    using char_type = charT;
    using const_iterator = typename
        basic_string_view<charT>::const_iterator;
    using iterator = const_iterator;

  private:
    iterator begin_;                                    // exposition only
    iterator end_;                                      // exposition only
    enum indexing { unknown, manual, automatic };       // exposition only
    indexing indexing_;                                 // exposition only
    size_t next_arg_id_;                                // exposition only
    size_t num_args_;                                   // exposition only

  public:
-   constexpr explicit basic_format_parse_context(basic_string_view<charT>
        fmt,
-                                                 size_t num_args = 0)
        noexcept;
+   constexpr explicit basic_format_parse_context(basic_string_view<charT>
        fmt) noexcept;
    basic_format_parse_context(const basic_format_parse_context&) = delete;
    basic_format_parse_context& operator=(const basic_format_parse_context&)
        = delete;

    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr void advance_to(const_iterator it);

    constexpr size_t next_arg_id();
    constexpr void check_arg_id(size_t id);

+   template<class... Ts>
+     constexpr void check_dynamic_spec(size_t id);
+   constexpr void check_dynamic_spec_integral(size_t id);
+   constexpr void check_dynamic_spec_string(size_t id);
  };
}
```

Remove the constructor:

```
constexpr explicit basic_format_parse_context(basic_string_view<charT> fmt,
                                              size_t num_args = 0) noexcept;
```

2 *Effects*: Initializes `begin_` with `fmt.begin()`, `end_` with `fmt.end()`, `indexing_` with `unknown`, `next_arg_id_` with `0`, and `num_args_` with ~~`num_args`~~ 0. [*Note 1:* Any call to `next_arg_id`, `check_arg_id`, or `check_dynamic_spec` on an instance of `basic_format_parse_context` initialized using this constructor is not a core constant expression. — *end note*]

And then add at the bottom:

```
constexpr void check_arg_id(size_t id);
```

9 *Effects*: If `indexing_ != automatic`, equivalent to:

```
if (indexing_ == unknown)
  indexing_ = manual;
```

10 *Throws*: `format_error` if `indexing_ == automatic` which indicates mixing of automatic and manual argument indexing.

11 *Remarks*: Call expressions where `id >= num_args_` are not core constant expressions ([expr.const]).

```
template<class... Ts>
  constexpr void check_dynamic_spec(size_t id);
```

12 *Mandates*: The types in `Ts...` are unique. Each type in `Ts...` is one of `bool`, `char_type`, `int`, `unsigned int`, `long long int`, `unsigned long long int`, `float`, `double`, `long double`, `const char_type*`, `basic_string_view<char_type>`, or `const void*`.

13 *Remarks*: Call expressions where `id >= num_args_` or the type of the corresponding format argument (after conversion to `basic_format_arg<Context>`) is not one of the types in `Ts...` are not core constant expressions ([expr.const]).

```
constexpr void check_dynamic_spec_integral(size_t id);
```

14 *Effects*: Equivalent to:

```
check_dynamic_spec<int, unsigned int, long long int, unsigned long long int>
        (id);

constexpr void check_dynamic_spec_string(size_t id);
```

15 *Effects*: Equivalent to:

```
check_dynamic_spec<const char_type*, basic_string_view<char_type>>(id);
```

§ 4 Acknowledgements

Thanks to Tim Song for discussing the issues and helping with the wording. Thanks to Victor Zverovich for having already solved the problem.

5 References

[P2216R3] Victor Zverovich. 2021-02-15. std::format improvements.
https://wg21.link/p2216r3

[P2757R0] Barry Revzin. 2023-01-08. Type checking format args.
https://wg21.link/p2757r0